

Code documentation for the paper on the system size and energy dependence of near-side jet-like correlations

Christine Nattrass (Yale, nattrass@rhig.physics.yale.edu),
Jana Bielcikova (NPI, jana.bielcikova@ujf.cas.cz),
Jörn Putschke (Yale, joern.putschke@yale.edu),
Helen Caines (Yale, helen.caines@yale.edu)

October 9, 2010

Code for this analysis is stored in `$CVSROOT/offline/paper/psn0530/`. The basic code structure is outlined in Figure 1, Figure 2, Figure 3, and Figure 4. There are six different libraries used. `StHighPtTree`, `StHighPtLoop`, `StHighPtEvaluator`, and `StHighPtTools` libraries contain classes and functions which can be used by multiple different analyses and `StAnalysis` and `StAnalysisTools` libraries contain classes and functions which are used only for the di-hadron correlation analysis.

Numerical codes were used for particle identification. The codes used in this analysis are defined in Table 1. In addition, sometimes classes of particles were studied separately to improve CPU efficiency. These classes of particles were given numerical codes which are defined in Table 2.

1 `StHighPtTree`, `StHighPtLoop`, `StHighPtEvaluator`, and `StHighPtTools` libraries

The `StHighPtTree`, `StHighPtLoop`, `StHighPtEvaluator`, and `StHighPtTools` libraries are diagramed in Figure 1 and Figure 2. Each function or class is briefly defined. `StHighPtTree` comprises container classes used in `TTrees` for the analysis. `StHighPtLoop` contains `StMakers` which can be used to loop

Table 1: Codes used for particle identification. Additional numbers were reserved for other particles these particles were later removed from the analysis.

Number	particle
0	h
1	Λ
2	$\bar{\Lambda}$
3	K_S^0
4	Ξ^-
5	Ω^-
7	π^+
8	π^-
9	p
10	$\bar{p}$
11	K^+
12	K^-
13	e^+
14	e^-
18	Ξ^+
18	Ω^+

Table 2: Codes used for classes of trigger and associated particles.

Number	particles
0	h
1	Λ , $\bar{\Lambda}$, and K_S^0
2	Ξ^- , Ξ^+ , Ω^- , and Ω^+
3	dE/dx identified particles ($\pi^\pm$, p , $\bar{p}$, $K^\pm$, $e^\pm$)

over TTrees or MuDsts. The primary class in this library is AnalysisMaker, which is a utility class for other analyses. It manages all track and event cuts and passes only events and tracks which pass quality cuts the next maker. TTreeMaker uses AnalysisMaker to create a TTree from a MuDst; this TTree can also be read by AnalysisMaker. StHighPtEvaluator contains classes which evaluate the quality of events and tracks. StHighPtTools con-

tains tools which are used by multiple other functions.

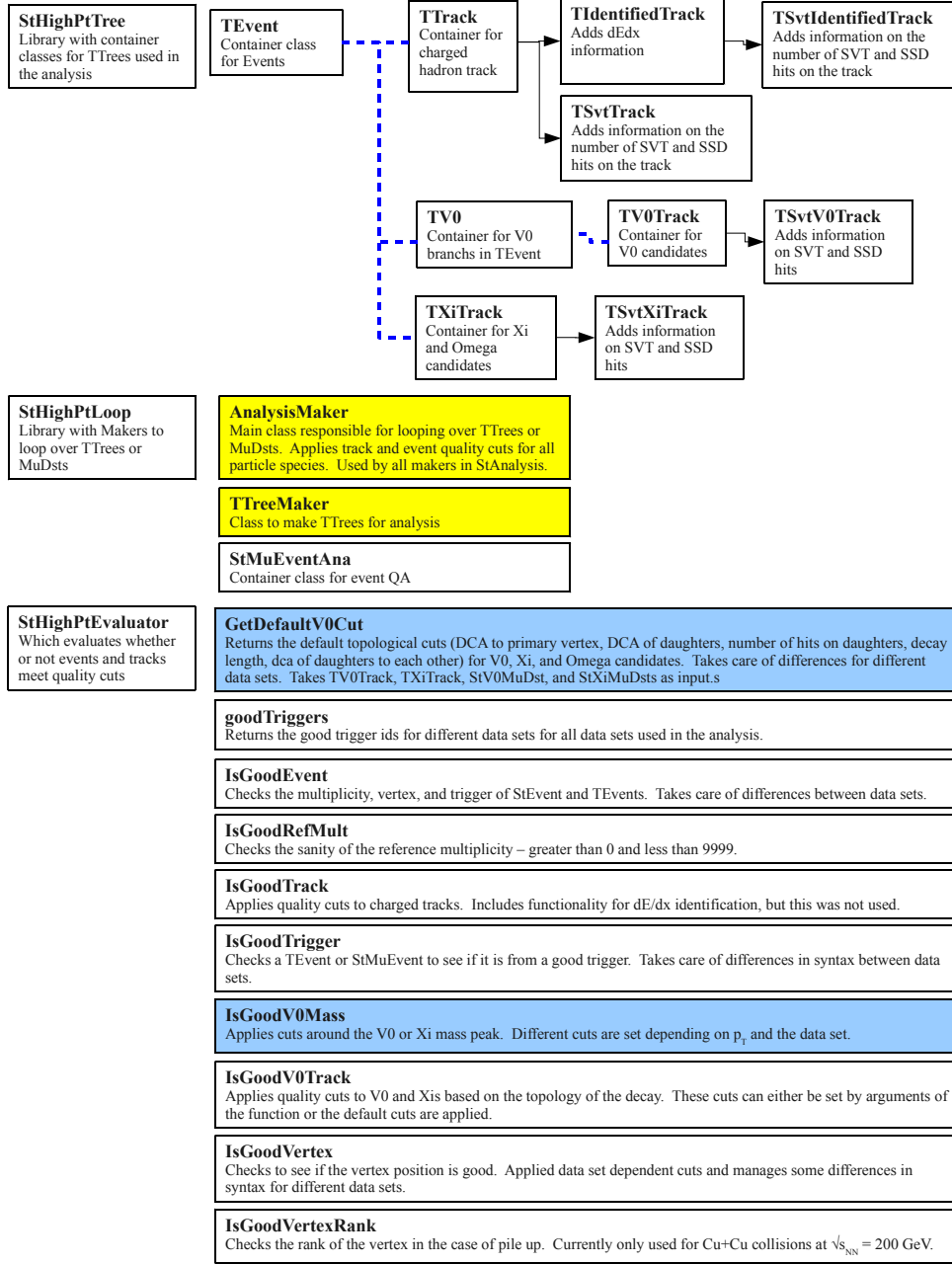


Figure 1: First part of the code diagram for the StHighPt libraries. Arrows indicate inheritance, blue dashed lines indicate objects held in a container class, yellow boxes indicate StMakers, and blue boxes indicate functions which have macros checked in to demonstrate these classes.

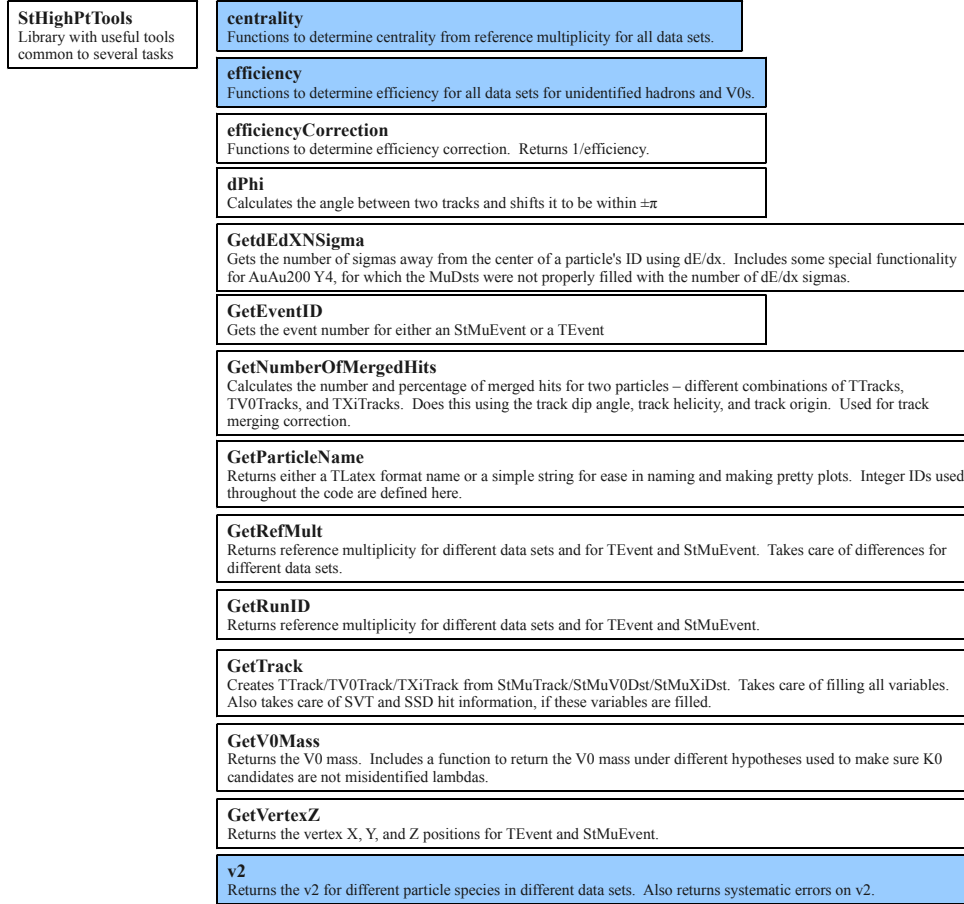


Figure 2: Second part of the code diagram for the StHighPt libraries. Arrows indicate inheritance, blue dashed lines indicate objects held in a container class, yellow boxes indicate StMakers, and blue boxes indicate functions which have macros checked in to demonstrate these classes.

2 StAnalysis and StAnalysisTools libraries

The StAnalysis and StAnalysisTools libraries are diagramed in Figure 3 and Figure 4. StAnalysis contains makers. CorrelationMaker is the maker used for the di-hadron correlation analysis. PhiMaker, RawSpectraMaker, and QAMaker are makers for quality assurance at high- p_T and are provided for convenience. V0CutTuner, XiCutTuner, OmegaCutTuner, and XiNtupleMaker are used for tuning the geometric cuts of V^0 s, Ξ^- , and Ω^- . StAnalysisTools contains functions which are used by the makers in StAnalysis. The primary class in this library is CorrelationSummary, which is used to contain all of the relevant information about a di-hadron correlation.

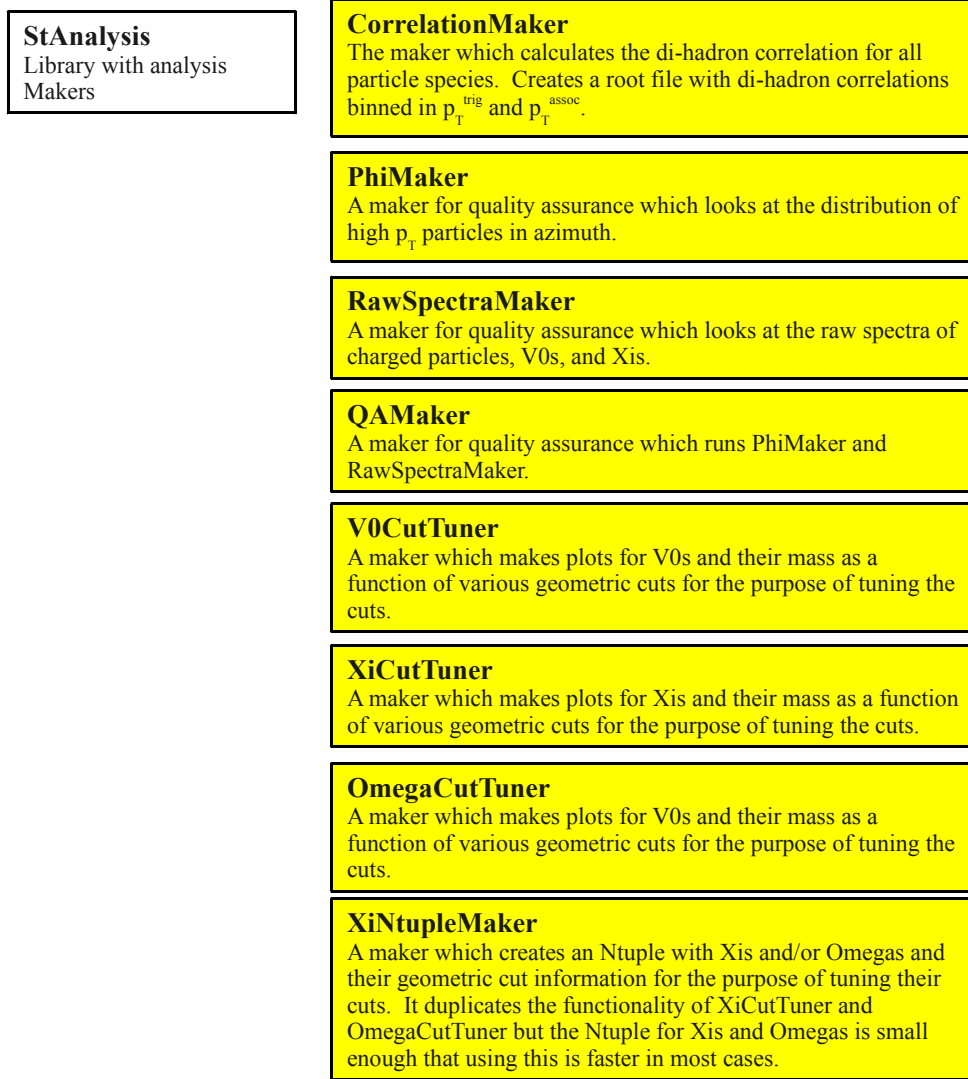


Figure 3: First part of the code diagram for the StAnalysis libraries. Arrows indicate inheritance, blue dashed lines indicate objects held in a container class, yellow boxes indicate StMakers, and blue boxes indicate functions which have macros checked in to demonstrate these classes.

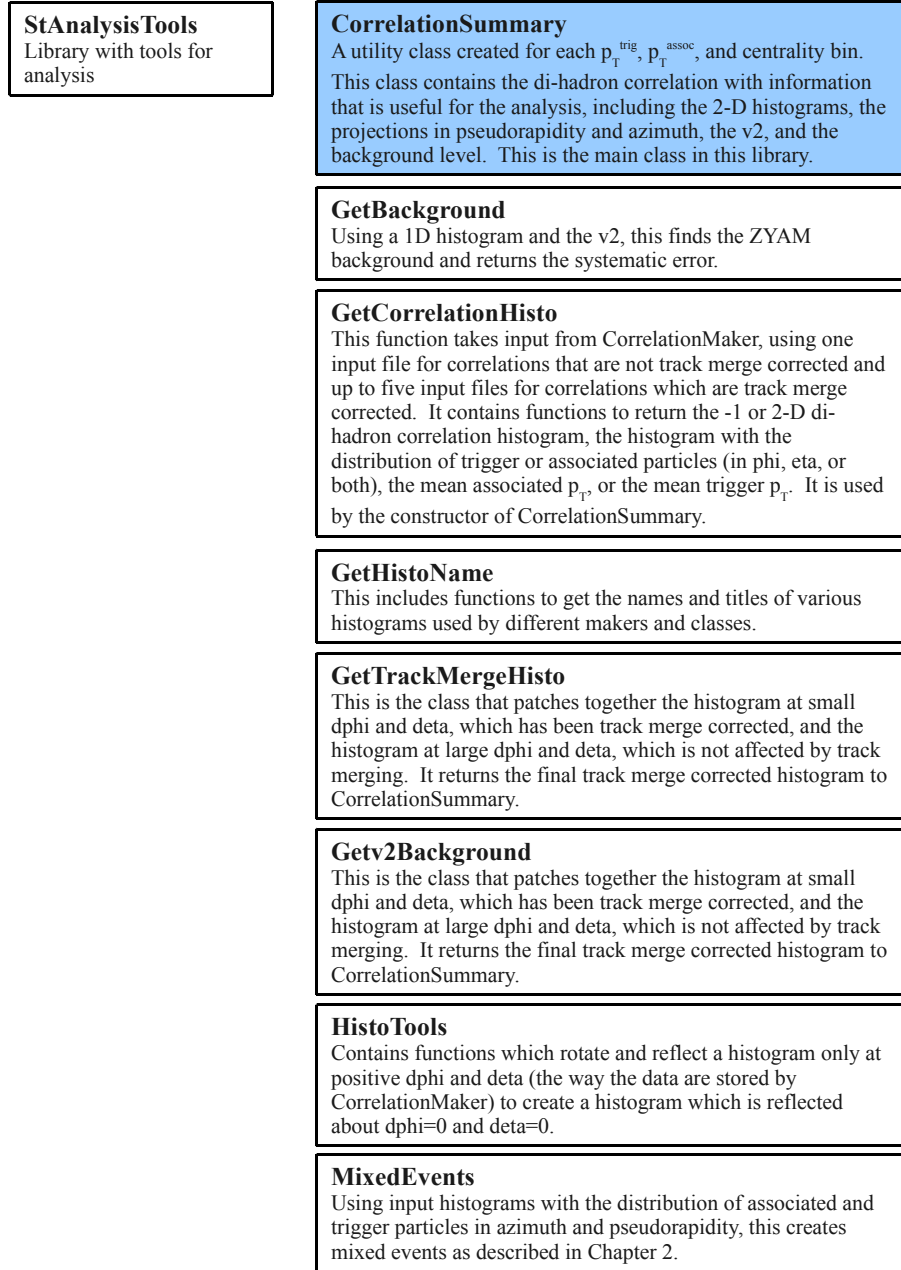


Figure 4: Second part of the code diagram for the StAnalysis libraries. Arrows indicate inheritance, blue dashed lines indicate objects held in a container class, yellow boxes indicate StMakers, and blue boxes indicate functions which have macros checked in to demonstrate these classes.

3 Sample macros

Sample macros for each maker and for a selection of functions are given in the macros directory. A brief description of how to use each macro is given. As described in the next section, these macros are not all needed to do the analysis, however, the additional macros are provided for future use.

1. **macros/test/testtools.C** Takes no arguments. Tests efficiency, efficiencyCorrection, centrality, and v2 and prints the result to the screen.
2. **macros/test/TestIsGoodV0Mass.C** Takes no arguments. Tests some of the functionality of IsGoodV0Mass and prints the result to the screen.
3. **macros/test/TestGetDefaultV0Cut.C**,
macros/test/TestGetDefaultXiCut.C Take no arguments. Print the default cuts out to the screen.

4. **macros/test/TestCorrelationSummary.C**

Example:

```
root4star -b -q macros/test/TestCorrelationSummary.C  
'(" AuAu200", "tmp/", 1.5, 6.0, 3.0, 6.0)'
```

This macro takes output files from CorrelationMaker and makes a CorrelationSummary. The arguments are the data set, the output directory for pictures, the minimum associated p_T , the maximum associated p_T , the minimum trigger p_T , and the maximum trigger p_T .

5. **macros/test/TestCorrelationSummaryTrackMergeCorrected.C**

Example:

```
root4star -b -q macros/test/TestCorrelationSummaryTrackmergeCorrected.C  
'(" AuAu200", "tmp/", 1.5, 6.0, 3.0, 6.0)'
```

This macro takes output files from CorrelationMaker and makes a CorrelationSummary with the track merging correction applied. The arguments are the data set, the output directory for pictures, the minimum associated p_T , the maximum associated p_T , the minimum trigger p_T , and the maximum trigger p_T .

6. **macros/MakeCuCu200P06ibTrees.C**

Example:

```
root4star -b -q macros/MakeCuCu200P06ibTrees.C'("inputMuDst.root", "output.root")'
```

Creates a TTree from a MuDst using TTreeMaker and AnalysisMaker.
This is written to run over 200 GeV Cu+Cu collisions.

7. **macros/QA/RunQAMaker.C**

Example:

```
root4star -b -q macros/QA/RunQAMaker.C
'("inputMuDst.root","output.phi.root","output.pt.root")'
```

This macro runs QAMaker, which in turn runs PhiMaker and RawSpectraMaker. In the background, it uses AnalysisMaker to filter tracks and events. The output is two root files for quality assurance of high- p_T particles.

8. **macros/geoCuts/RunOmegaCutTuner.C**

Example:

```
root4star -b -q macros/geoCuts/RunOmegaCutTuner.C
'("inputTTree.root","output.root")'
```

Runs OmegaCutTuner with an input TTree and creates an output file with histograms of omega mass peaks.

9. **macros/geoCuts/RunV0CutTuner.C**

Example:

```
root4star -b -q macros/geoCuts/RunV0CutTuner.C'("inputTTree.root","output.root")'
```

Runs V0CutTuner with an input TTree and creates an output file with histograms of V^0 mass peaks.

10. **macros/geoCuts/RunXiCutTuner.C**

Example:

```
root4star -b -q macros/geoCuts/RunXiCutTuner.C'("inputTTree.root","output.root")'
```

Runs XiCutTuner with an input TTree and creates an output file with histograms of Ξ^- mass peaks.

11. **macros/geoCuts/RunXiNtupleMaker.C**

Example:

```
root4star -b -q macros/geoCuts/RunXiNtupleMaker.C'("inputTTree.root","output.root")'
```

Runs XiNtupleMaker with an input TTree and creates an output file with an Ntuple with information about the geometric cuts for Ξ^- and/or Ω^- .

12. **macros/correlations/RunCorrelationMaker.C**

Example:

```
root4star -b -q macros/correlations/RunCorrelationMaker.C
'("inputTTree.root","output.root",0,0)'
```

This runs CorrelationMaker using a TTree as input and creating a file with di-hadron correlation histograms. The last two arguments are particle classes defined in Table 2.

13. **macros/correlations/RunCorrelationMakerForTrackMerging.C**

Example:

```
root4star -b -q macros/correlations/RunCorrelationMakerForTrackMerging.C
'("inputTTree.root","output.root",0,0,1,1)'
```

This runs CorrelationMaker using a TTree as input and creates a file with di-hadron correlation histograms but only for small $\Delta\eta$ and $\Delta\phi$ for one combination of trigger and associated helicities at a time. It also eliminates track pairs where the two tracks share more than 10% of their hits, as described in Chapter 2. The third and fourth arguments are particle classes defined in Table 2. The fifth and sixth arguments are the helicity of the trigger and associated particles, respectively. Note that it is possible to request an impossible helicity combination and then the histograms will be empty. In this example, unidentified trigger and associated particles are requested, each with a positive helicity.

14. **macros/correlations/RunCorrelationMakerForEventMixingCuCu200.C**

Example:

```
root4star -b -q macros/correlations/RunCorrelationMakerForEventMixingCuCu200.C
'("inputTTree.root","output.root",0,0,1,1)'
```

This runs CorrelationMaker using a TTree as input and creates a file with di-hadron correlation histograms from mixed events for only for small $\Delta\eta$ and $\Delta\phi$ for only one combination of trigger and associated helicities. It also eliminates track pairs where the two tracks share more than 10% of their hits, as described in Chapter 2. The arguments are the same as for RunCorrelationMakerForTrackMerging.C.

15. **macros/correlations/RunCorrelationMakerForEventMixingCuCu200InPieces.C**

Example:

```
root4star -b -q macros/correlations/
RunCorrelationMakerForEventMixingCuCu200InPieces.C
'(0, "inputTTree.root","output.root",0,0,1,1)'
```

This does the same thing as RunCorrelationMakerForEventMixingCuCu200.C,

however, it will run over only a subset of the input TTree. This is necessary for event mixing to reduce the duration of jobs. This macro is written to run over bunches of 10,000 events. Smaller subsets can be chosen by editing the macro. The arguments are in the same order as RunCorrelationMakerForTrackMerging.C but there is an additional first argument which is an integer defining which part of the TTree to run over. The argument 0 runs over the first 10,000 events.

4 Analysis procedure

1. Create TTrees from MuDsts using MakeCuCu200P06ibTrees.C
2. Create output files with di-hadron correlations using RunCorrelationMaker.C. At this point correlations which are not corrected for track merging are available and can be studied with TestCorrelationSummary.C.
3. Create output files with di-hadron correlations at small $\Delta\phi$ and small $\Delta\eta$ using RunCorrelationMakerForTrackMerging.C for each trigger and associated helicity combination. Create corresponding mixed event files using RunCorrelationMakerForEventMixingCuCu200.C or RunCorrelationMakerForEventMixingCuCu200InPieces.C for each trigger and associated helicity combination. For h-h, Ξ^- -h, and Ω^- -h correlations this corresponds to an additional eight files, since each trigger and each associated particle can have a helicity of either +1 or -1. For V^0 -h and h- V^0 correlations there are only four files created at this stage since the V^0 can only have a helicity of 0.
4. Di-hadron correlations can be corrected for track merging are created by using the output file created in the second step and the four or eight input files from the third step to create a track merging corrected di-hadron correlation using TestCorrelationSummaryTrackMergeCorrected.C. CorrelationSummaries can be written to a root file and used later or re-fit.